UPSIDE DOWN / HACKATHON STUDIO

Laptop B.
One clear role.

Frontend & experience

Make the invisible teamwork visible.

ASSIGNMENT

P2

OWNED PATHS

web/**

DEMO BRANCH

demo/p2-frontend

YOUR HANDOFF PACK

1. Your role prompt
2. Common context.md
3. Complete supplied ARCHITECTURE.md
4. Complete supplied CONTRACT.md

Fixed reference plan. Device assignments and planning are simulated. This document does not claim that agents executed code or connected to other laptops. The supplied documents describe AgentSync, not a migration of the running Upside Down application.

P2 - Laptop B: Frontend & experience

Read context.md, ARCHITECTURE.md and CONTRACT.md in full before starting. They are included in your PDF.

Your objective

Make the invisible teamwork visible.

Owned files

• web/**

Branch

demo/p2-frontend (illustrative assignment; no branch is created by this demo)

Deliverables

1. Build agents, tasks, activity, conflicts and sync panels
2. Start with labeled fixtures, then connect to the public API
3. Refetch affected lists on state.changed SSE events
4. Handle loading, empty, offline, errors and manual conflict review

Ownership boundary

Do not read databases, edit server internals or expose GitHub tokens.

Handoff

Consume P3's response fixtures and /api endpoints; demonstrate the P4 scenario.

Acceptance

Run the tests assigned to P2 in CONTRACT.md section 12. Report the exact commands, observed results, changed files, interface assumptions and outstanding blockers. Keep fixtures labeled DEMO DATA. Do not claim remote execution or unrun tests.

Integration

Use the supplied API shapes without renaming. Request explicit handoffs for other owners' files. Provide a small reviewable commit on your assigned branch. Integrate only after shared interface agreement and local checks; do not auto-merge. GitHub sync is opt-in, server-side and best effort.

Preserve the existing product

This pack describes the AgentSync reference project. Do not replace the running Upside Down application or its contract. If asked to implement, use a separate approved project directory and verify scope with the human.

context.md - AgentSync shared project context

One consistent problem

Build AgentSync: a local-first workspace where four hackathon laptops coordinate coding tasks, see API contract changes, resolve conflicting edits, and queue GitHub updates when the internet drops.

What this handoff is

This is a predefined hackathon demo plan inside Upside Down. Uploads select local file metadata only; the uploaded contents are not parsed or sent anywhere. Every input produces the same supplied ARCHITECTURE.md and CONTRACT.md. Planning, device identities, tool choices and assignment progress are simulated; no LLM, real laptop discovery or remote code execution is claimed.

Read order on every laptop

1. context.md (this common coordinator brief)
2. ARCHITECTURE.md (the supplied source, unchanged)
3. CONTRACT.md (the supplied source, unchanged)
4. Your assigned P1, P2, P3 or P4 prompt

Four laptops, four boundaries

Laptop A / P1: coordinator and SQLite.

Laptop B / P2: React frontend.

Laptop C / P3: HTTP API, shared schemas and SSE.

Laptop D / P4: bridge, GitHub adapter and QA.

The architecture's reproducible event scenario uses three active simulated agents; the fourth laptop owns the dashboard. Four laptop assignments do not mean four live connected machines.

Build order

Freeze the shared interface and fixtures first. P1 implements CoordinatorService while P3 starts against a fake service. P2 uses fixtures before the API is ready. P4 posts only through HTTP. Integrate at the architecture's G0-G4 gates. Persist before broadcast, retain event IDs on retries, and keep GitHub optional.

Demo story

An API contract changes; one consumer acknowledges it; another is explicitly incompatible and receives a conflict. Disable public internet while retaining localhost/LAN. Local events continue and cloud jobs queue. Reconnect and sync if configured. A human resolves conflicts. There is no automatic merge.

Integration rules

Use exactly the supplied contract's paths, record shapes, enums, errors and service signatures. One owner per folder. Request a handoff before editing another owner's files. Use additive changes and obtain affected-owner approval for incompatible changes. Never infer incompatibility from natural language. Never expose secrets or claim a remote sync without a real remote URL.

Running the existing Upside Down demo

This demo is an additive frontend presentation. Existing Upside Down APIs retain their current UD-IC contract. The supplied AgentSync V1 specification is a teaching / handoff artifact, not a replacement for the running application's backend contract. Do not overwrite or migrate the existing website to implement these prompts.

Prompt access

The website's laptop selector demonstrates role-scoped views; it is not authentication. All bundled documents are public demo artifacts. Real multi-device authorization and execution are outside this prototype. A host may inspect all packs; a selected laptop sees its own role prompt plus the three common documents.

ARCHITECTURE.md — AgentSync (working name)

Upside Down Hackathon · PS-03-first, with PS-02-style conflict awareness · 4 developers · 15-hour implementation window

Working name is a placeholder. Inspiration: `azizu06/hackathon-agent-board`. We are **not** claiming its features are implemented in our project until tested.

1. Product in one sentence

A local-first control plane where multiple coding agents working on one repository publish progress, discover contract changes, avoid conflicting edits, keep functioning without internet, and synchronize an auditable activity digest to GitHub when connectivity returns.

2. The demo we must deliver

1. Open a local dashboard showing three simulated agents working on one repository.

2. Agent A modifies a shared API contract (for example `GET /api/tasks` now requires `status`).

3. The system emits a `contract.changed` event and flags Agents B and C as potentially affected.

4. Agent B acknowledges the change; an incompatible edit by Agent C creates a visible conflict requiring human resolution.

5. Turn off **internet access** while retaining localhost/LAN access. Agents continue communicating with the local server; events are stored durably; GitHub-sync jobs queue.

6. Restore internet. Trigger sync and show each queued item completing once, with a GitHub link if configured.

7. Show a clear chronological audit trail. **No automatic merge** in the MVP.

Honesty boundary: Offline means no cloud/internet connection, NOT that independent devices communicate with zero local network. Single-laptop localhost is the reliable demo fallback. The agents can be simulators/CLI scripts, not autonomous commercial LLM integrations.

3. Scope: Must / Should / Cut

Priority	Deliverable
MUST	Local coordinator with persisted events, agents, tasks and conflict flags
MUST	API contract frozen before independent implementation
MUST	Dashboard: agent status, tasks, feed, conflicts, sync queue
MUST	`POST /api/events` with event IDs and idempotency
MUST	Live updates via Server-Sent Events (SSE) with polling fallback
MUST	CLI/event simulator used for reproducible demo
MUST	File/contract-change notification, initially rule-based from explicit event payload
MUST	Offline-capable local activity and persistent cloud-sync queue
SHOULD	GitHub issue/comment sync when token and network are available
SHOULD	File ownership and simple conflict detection
SHOULD	File watcher for actual local repo changes
CUT	Full autonomous agent orchestration, AI conflict adjudication, auto-merging, peer-to-peer mesh, multiple organizations, auth, production deployment

MVP differs from a basic agent message board through the combined live dependency warning, offline event persistence, queue replay and conflict review workflow. These are planned claims, not verified results.

4. Four people, one owner per boundary

Owner	Module	Owns	Must NOT change	Completion evidence
P1 — Backend / coordinator (you)	`server/src/core/**`, `server/src/db/**`	SQLite schema, persistence, agent/task registry, conflict rules, queue/outbox, coordinator service	API route handlers, UI, GitHub adapter	Unit tests for append, dedupe, conflicts, restart durability
P2 — Frontend / UX	`web/**`	React dashboard, agent cards, event stream, task board, conflicts and sync status, demo-friendly visuals	Server internals and schemas	Screenshots + browser demo driven by fixtures and then API
P3 — API / realtime	`server/src/http/**`, `shared/**`	Routes, runtime request validation, errors, SSE, `openapi.yaml`/fixtures, CORS, mocks	Database internals, UI, GitHub adapter	`curl`/API contract tests, SSE demo, validation coverage
P4 — Agent integration / QA	`bridge/**`, `server/src/integrations/**`, `scripts/**`, `tests/e2e/**`	CLI bridge, simulated agents, optional filesystem watcher, GitHub sync adapter, scenario automation, E2E	Coordinator logic, API routes, UI	One-command scenario passes offline/reconnect and restart tests

No shared-file editing without notifying the file owner. Each developer works on a separate branch; the integrator does not become the only person responsible for merges. Owners merge their own tested PRs at planned checkpoints.

Critical interface between P1 and P3

P3 calls the `CoordinatorService` interface in `shared/service-contract.ts` implemented by P1. P3 starts against an in-memory fake implementing that same interface. P1 does not implement HTTP. Every API calls a method on the service; only P1 touches database tables.

Critical interface between P3 and P2/P4

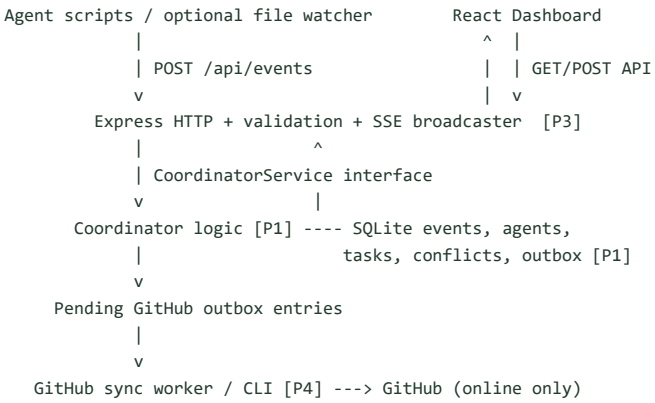
P3 publishes `shared/schemas.json`, sample JSON response fixtures and API routes defined in `CONTRACT.md` within the first hour. P2 uses fixtures until the server is ready. P4 posts events through the public HTTP API; it does not import coordinator internals.

5. Reference structure

```
agentsync/
├── ARCHITECTURE.md
├── CONTRACT.md
├── README.md
├── package.json               # npm workspace scripts
├── shared/
│   ├── schemas.json            # event/request/response definitions; P3
│   ├── service-contract.ts     # interface jointly frozen P1 + P3
│   └── fixtures/               # versioned mocks; P3
├── server/
│   ├── src/
│   │   ├── index.ts           # bootstrap wiring; P3
│   │   ├── core/              # coordinator logic; P1
│   │   ├── db/                # SQLite + migrations; P1
│   │   ├── http/              # Express routes, SSE; P3
│   │   └── integrations/      # GitHub adapter and outbox worker; P4
│   └── data/                  # local SQLite DB; gitignored
├── web/
│   └── src/                   # React/Vite; P2
├── bridge/                    # CLI simulator + optional watcher; P4
├── scripts/                   # setup, seed, demo, reset; P4
└── tests/
    ├── contracts/             # P3
    ├── core/                  # P1
    └── e2e/                   # P4
```

Suggested stack: TypeScript, React + Vite, Node + Express, SQLite (`better-sqlite3` if compatible with the installed Node version), SSE, `zod` or equivalent validation, native `fetch`, and optionally `chokidar` for file watching. No extra infra required for local demo.

6. Logical data flow



• **Persist first, then broadcast.** A successful event POST means the event is durably accepted.

• **Source of truth:** local SQLite. GitHub is a synchronized outward projection, never required for local operation.

• **Ordering:** server assigns increasing `sequence`; client event timestamps are for display only.

• **Idempotency:** `eventId` uniquely identifies an event; a repeated POST returns the original record without duplicate side effects.

• **Realtime:** SSE for dashboard notifications; `GET /api/events?afterSequence=N` as recovery/poll fallback.

• **Conflict detection:** deterministic rules first: competing open file claims; contract-change dependency overlap; incompatible explicit contract versions. A conflict is a warning, not proof of a merge conflict.

• **Security:** bind server to `127.0.0.1` for a single-device demo; no production security promises. Store GitHub token only in backend environment variables, never in client code or JSON payloads.

7. Build schedule (15-hour engineering window)

Time from kickoff	P1	P2	P3	P4	Gate
0:00–0:45	Approve service methods + schema	Wireframe five panels	Freeze endpoint names, sample fixtures	Create three-agent scenario script	**G0:** `CONTRACT.md` approved by all
0:45–3:00	SQLite + event append/dedupe	Layout with mock data	Implement validation + mocked routes + SSE	CLI emitter + scripted payloads	**G1:** mock full path works
3:00–6:00	State, queue, conflict rules	Connect live API + SSE	Replace mock service with P1 service	GitHub outbox worker stub + tests	**G2:** live dashboard with local writes
6:00–9:00	Persistence hardening	Conflict review + UX polish	Contract test suite + recovery API	Offline/reconnect scenario + optional watcher	**G3:** demo can run without internet
9:00–12:00	Fix core bugs only	Accessibility/responsive polish	End-to-end API checks	Optional GitHub live sync, or mock-mode output	**G4:** entire scripted walkthrough passes
12:00–13:30	Integration support	Demo walkthrough polish	Fix blockers only	Failure injection + demo seed/reset	**Feature freeze** at 13:30
13:30–15:00	Test + presentation	Test + presentation	Test + presentation	Run demo twice + prepare fallback	**Code freeze** at scheduled cutoff

Hard deadline override: if the event ends with coding freeze at **08:30** and submission at **08:45**, stop code changes at 08:30 regardless of the relative schedule; leave the last 15 minutes for packaging and submission.

8. Integration gates — observable, not verbal

• **G0** Four approvals, endpoint examples load, schema decisions recorded.

• **G1** `npm run demo:seed` emits three agents' events into mocked API; UI renders fixtures.

• **G2** Real event POST visible in UI within ~2 seconds on local machine (demo target, not a guaranteed SLA).

• **G3** Disable internet; event persists; restart server; event still exists and outbox says `pending`.

• **G4** Reconnect internet; if configured, GitHub sync transitions `pending → synced`; repeated sync does not duplicate created remote issues/comments.

• **Final** Fresh clone/install/start + one command to reproduce demo, with an offline/mock fallback.

9. Team working agreement

1. Each task has an owner, output artifact and short due time; no ambiguous “research something interesting” tasks.

2. Every 60–90 minutes, each person posts **commit/PR, evidence, blocker and next step**.

3. If blocked >20 minutes, raise it immediately. If a commitment is missed twice, cut that feature or reassign a specific bounded task.

4. **Contract changes require all affected owners' approval.** Additive fields are preferred; never silently rename endpoints or event types.

5. Keep `main` runnable. PRs are small; avoid simultaneous edits to someone else's module.

6. The leader should **not** silently absorb unfinished tasks. Reduce scope before expanding anyone else's workload.

7. If live GitHub sync threatens the working demo, disable it and demonstrate persisted outbox with clearly labeled simulation.

10. Definition of done

• Three distinct agent IDs can send events and appear on dashboard.

• Change to a shared contract visibly warns at least one subscribed agent.

• A detected competing edit/claim creates a reviewable conflict; user can resolve it with a note.

• Local operations survive simulated internet loss and server restart.

• Outbox sync status and audit timeline are visible; duplicate events do not duplicate records.

• E2E demo script and README explain exactly what is simulated versus real.

CONTRACT.md — AgentSync V1 Integration Contract

Freeze this document in the first 45 minutes. All four contributors build against it. Proposed interface specification, not a claim that the code already implements it.

0. Non-negotiable ownership

• **P1** owns coordinator and SQLite implementation (`server/src/core`, `server/src/db`). Never implement Express routes.

• **P2** owns React UI (`web`). Never directly read `.db`, JSON storage, or GitHub tokens.

• **P3** owns HTTP handlers, validation, shared schemas, service interface and SSE (`server/src/http`, `shared`). Never query SQLite from route handlers.

• **P4** owns agent CLI, GitHub adapter, demo runner and E2E tests (`bridge`, `server/src/integrations`, `scripts`, `tests/e2e`). All agent writes pass through HTTP API.

PRs touching another person's owned code require explicit approval. Any incompatible change to this contract needs P1 + P3 approval and notification to P2 + P4.

1. System configuration

```
API_HOST=127.0.0.1
API_PORT=4000
VITE_API_BASE_URL=http://127.0.0.1:4000
DATABASE_PATH=./server/data/agentsync.sqlite
GITHUB_TOKEN=                # optional, server only
GITHUB_REPO=owner/repo       # optional, server only
GITHUB_SYNC_ENABLED=false    # opt-in
```

Endpoints use base `/api`. The API must send `Content-Type: application/json`. UI connects to SSE at `/api/stream`. For a 2-device LAN demo, explicitly set API_HOST to the host's LAN interface and update frontend API URL; do not expose outside trusted demo LAN.

2. Canonical IDs, enums and timestamps

• All IDs are stable, opaque strings (UUID recommended), except `sequence` which is a server-assigned increasing integer.

• Timestamps: ISO-8601 UTC with trailing `Z`.

• `agent.status`: `idle` | `working` | `blocked` | `offline`.

• `task.status`: `todo` | `in_progress` | `blocked` | `done`.

• `conflict.status`: `open` | `resolved` | `dismissed`.

• `outbox.status`: `pending` | `syncing` | `synced` | `failed`.

• `event.type`: `agent.registered` | `agent.status_changed` | `task.updated` | `file.claimed` | `file.released` | `contract.changed` | `message.sent` | `conflict.detected` | `conflict.resolved`.

• `event.source`: `cli` | `watcher` | `ui` | `system`.

• `filePath` is a **repository-relative POSIX path** (`src/api/types.ts`), never an absolute path. Reject paths containing `..` segments.

• Do not rename these values after G0.

3. Canonical JSON records

Agent

```
{
  "id": "agent-a",
  "name": "Backend Agent",
  "role": "backend",
  "status": "working",
  "lastSeenAt": "2026-11-04T00:00:00Z",
  "ownedPaths": ["server/src/core/**"]
}
```

Task

```
{
  "id": "task-1",
  "title": "Implement task API",
  "ownerAgentId": "agent-a",
  "status": "in_progress",
  "dependsOnTaskIds": [],
  "affectedPaths": ["server/src/http/tasks.ts"],
  "updatedAt": "2026-11-04T00:00:00Z"
}
```

Event (server response)

```
{
  "eventId": "evt-contract-001",
  "sequence": 42,
  "type": "contract.changed",
  "agentId": "agent-a",
  "source": "cli",
  "occurredAt": "2026-11-04T00:00:00Z",
  "receivedAt": "2026-11-04T00:00:01Z",
  "payload": {
    "contractId": "tasks-api",
    "oldVersion": "v1",
    "newVersion": "v2",
    "changedPaths": ["shared/openapi.yaml"],
    "summary": "GET /api/tasks now supports required status filter",
    "affectedAgentIds": ["agent-b"]
  }
}
```

Conflict

```
{
  "id": "conflict-001",
  "kind": "contract_dependency",
  "status": "open",
  "filePath": "shared/openapi.yaml",
  "agentIds": ["agent-a", "agent-b"],
  "eventIds": ["evt-contract-001"],
  "reason": "Consumer agent is using an older contract version",
  "createdAt": "2026-11-04T00:00:01Z",
  "resolutionNote": null,
  "resolvedAt": null
}
```

Outbox entry

```
{
  "id": "sync-001",
  "eventId": "evt-contract-001",
  "status": "pending",
  "attempts": 0,
  "lastError": null,
  "remoteUrl": null,
  "updatedAt": "2026-11-04T00:00:01Z"
}
```

Do not return secrets or absolute local paths in public API responses.

4. HTTP endpoints (P3 owns route implementations)

Method	Path	Purpose	Request	Success
GET	`/api/health`	Server health	—	`200 { "ok": true, "mode": "local" }`
GET	`/api/agents`	List agent records	—	`200 { "items": Agent[] }`
POST	`/api/agents`	Register/update agent	Agent input, no `lastSeenAt`	`200 { "item": Agent }`
GET	`/api/tasks`	List tasks	—	`200 { "items": Task[] }`
POST	`/api/tasks`	Create task	Task input, server timestamps	`201 { "item": Task }`
PATCH	`/api/tasks/:id`	Edit task status/owner/title	Partial task fields	`200 { "item": Task }`
GET	`/api/events?afterSequence=0&limit=100`	Ordered event log	query	`200 { "items": Event[], "nextSequence": 42 }`
POST	`/api/events`	Append agent event idempotently	Event input	`201` or replay `200 { "item": Event, "duplicate": boolean }`
GET	`/api/conflicts`	List conflicts	optional `status`	`200 { "items": Conflict[] }`
PATCH	`/api/conflicts/:id`	Resolve/dismiss with note	`{ "status": "resolved", "resolutionNote": "..." }`	`200 { "item": Conflict }`
GET	`/api/outbox`	List queued sync items	—	`200 { "items": OutboxEntry[] }`
POST	`/api/sync`	Request one sync pass	`{}`	`202 { "accepted": true }`
GET	`/api/stream`	SSE live updates	`Accept: text/event-stream`	SSE stream

`POST /api/events` sample

```
POST /api/events
Content-Type: application/json
```

```
{
  "eventId": "evt-contract-001",
  "type": "contract.changed",
  "agentId": "agent-a",
  "source": "cli",
  "occurredAt": "2026-11-04T00:00:00Z",
  "payload": {
    "contractId": "tasks-api",
    "oldVersion": "v1",
    "newVersion": "v2",
    "changedPaths": ["shared/openapi.yaml"],
    "summary": "GET /api/tasks now requires status filter",
    "affectedAgentIds": ["agent-b"]
  }
}
```

Rules:

• Server supplies `sequence` and `receivedAt`; clients must not.

• Duplicate `eventId`: return original record as `200`, `duplicate: true`. Never duplicate conflicts/outbox items.

• An unknown `agentId`, illegal event type or malformed event-specific payload returns `422`.

• `limit` defaults to 100 and is capped at 200; `afterSequence` defaults to 0.

• `POST /api/agents` with an existing ID updates the existing agent rather than adding a duplicate.

• `POST /api/sync` enqueues/starts a sync pass; it must not block on GitHub API latency.

5. Event payload contracts (P1/P3/P4)

All events share `eventId`, `type`, `agentId`, `source`, `occurredAt`, `payload`.

Type	Required payload	Behavior
`agent.registered`	`{ "name": string, "role": string }`	audit registration
`agent.status_changed`	`{ "status": AgentStatus }`	update agent status
`task.updated`	`{ "taskId": string, "status": TaskStatus }`	update task state
`file.claimed`	`{ "filePath": string }`	record claim; if already claimed by another active agent create conflict
`file.released`	`{ "filePath": string }`	release the caller's claim
`contract.changed`	`{ "contractId": string, "oldVersion": string, "newVersion": string, "changedPaths": string[], "summary": string, "affectedAgentIds": string[] }`	create affected-agent notice, and conflict if compatibility is explicitly flagged
`message.sent`	`{ "toAgentId": string, "text": string }`	route visible message to recipient
`conflict.detected`	`{ "kind": string, "agentIds": string[], "reason": string, "filePath": string|null }`	system/bridge conflict alert
`conflict.resolved`	`{ "conflictId": string, "resolutionNote": string }`	close conflict; server validates permissions in future production scope

For `contract.changed`, optionally include `incompatibleAgentIds: string[]` to create a `contract_dependency` conflict for each listed affected agent. **Affected does not automatically mean incompatible.** The demo script must explicitly mark incompatibility or provide a deterministic version mismatch, not infer it from natural language.

6. SSE contract

`GET /api/stream` returns `text/event-stream`. Events have this form:

```
id: 42
event: state.changed
data: {"sequence":42,"entity":"event","entityId":"evt-contract-001"}
```

• `entity`: `event | agent | task | conflict | outbox`.

• When any entity changes, P3 emits one `state.changed` with `entityId`. For changes not backed by a new event, `sequence` is the most recent event sequence.

• P2/P4 react by refetching the affected GET endpoint. This avoids synchronizing full object schemas inside SSE messages.

• On SSE reconnect, the UI calls `GET /api/events?afterSequence=<lastSeenSequence>` and refreshes lists to recover missed updates.

• If SSE is not available, poll relevant GET endpoints every 2 seconds while demo is open.

7. CoordinatorService interface (P1 implements, P3 consumes)

```
export interface CoordinatorService {
  health(): Promise<{ ok: true; mode: 'local' }>;
  listAgents(): Promise<Agent[]>;
  upsertAgent(input: AgentInput): Promise<Agent>;
  listTasks(): Promise<Task[]>;
  createTask(input: TaskInput): Promise<Task>;
  updateTask(id: string, patch: TaskPatch): Promise<Task>;
  listEvents(afterSequence: number, limit: number): Promise<Event[]>;
  appendEvent(input: EventInput): Promise<{ item: Event; duplicate: boolean }>;
  listConflicts(status?: ConflictStatus): Promise<Conflict[]>;
  updateConflict(id: string, patch: ConflictPatch): Promise<Conflict>;
  listOutbox(): Promise<OutboxEntry[]>;
  requestSync(): Promise<void>;
  onChange(listener: (event: { sequence: number; entity: 'event'|'agent'|'task'|'conflict'|'outbox'; enti
  tyId: string }) => void): () => void;
}
```

P3 defines the concrete TypeScript types based on the JSON examples above. P1 returns domain objects, **never Express response objects**. P3 owns HTTP status mapping and validation. P4's worker may use a narrow `OutboxRepository` interface from P1 but must not reach into SQLite directly; P1 owns SQL and idempotent sync-state transitions.

8. Storage and sync guarantees

• Tables: `agents`, `tasks`, `events`, `file_claims`, `conflicts`, `outbox`.

• `events.event_id` has UNIQUE constraint; `events.sequence` is auto-increment PK.

• Outbox entries are created atomically with the event that requires cloud sync; `outbox.event_id` UNIQUE.

• Crash/restart retains accepted events, conflicts and pending outbox rows.

• P4 retries failures with attempts/status visible; no endless tight retry loop.

• GitHub sync is **best-effort**, not guaranteed exactly-once across a crash after remote creation. Embed `eventId` in GitHub issue/comment marker and check for it before posting to reduce duplicates.

• Without GitHub credentials or internet, show `pending` and explain that remote sync is unavailable; **never display a fake GitHub URL**.

9. UI acceptance contract (P2)

Five primary panels:

1. **Agents:** name, role, status, last seen, owned paths.

2. **Tasks:** title, owner, status and dependencies.

3. **Activity:** chronological typed events, filter by agent, latest first or oldest first toggle.

4. **Conflicts:** type, agents, affected file, reason, Resolve/Dismiss with note.

5. **Sync:** pending/synced/failed counts and manual `Sync now` button.

Required states: loading, empty, API error, SSE disconnected, internet-offline-but-local-server-online, and local-server-disconnected. Never represent loss of GitHub internet as a local server outage. Do not claim a conflict is auto-resolved. All sample fixtures must be visually labeled **DEMO DATA** until real events are received.

10. Agent bridge and GitHub sync contract (P4)

Bridge CLI examples:

```
npm run agent:demo -- --agent agent-a --scenario contract-change
npm run agent:demo -- --agent agent-b --scenario acknowledge
npm run demo:full
```

• Calls POST `/api/agents` once per agent, then POST `/api/events` with stable IDs.

• For true local file events, an optional file watcher can map file changes to a `file.claimed` or `contract.changed` signal; do not automatically declare breaking changes based solely on file modification.

• Simulator scenarios are clearly labeled as simulated agent activity, not autonomous AI agents.

• GitHub adapter uses server-side env token and `GITHUB_REPO`; disabled by default.

• If sync is enabled, publish a summary into an existing designated issue or create a dedicated issue using a stable marker; do not spam one issue per event by default.

11. Error format

All non-2xx JSON errors:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid event payload",
    "details": [{ "field": "type", "reason": "Unknown enum value" }]
  }
}
```

Codes: `VALIDATION_ERROR` (`400`/`422`), `NOT_FOUND` (`404`), `CONFLICT` (`409`), `INTERNAL_ERROR` (`500`), `SYNC_UNAVAILABLE` (`503`, only for a direct sync-preflight failure; normal queued offline sync returns `202`).

12. Contract acceptance tests — owners must run before merging

Test	Owner	Expected
POST valid event → GET event log	P1 + P3	event exists with sequence
POST identical event twice	P1 + P3	second response `duplicate: true`, no extra outbox/conflict
Two agents claim same file	P1	one open conflict with both agents
Change compatible contract	P1 + P4	alert only; no false conflict
Mark incompatible dependent agent	P1 + P4	open dependency conflict
SSE dashboard receives change	P2 + P3	UI updates without refresh
Disconnect public internet	P4	local events still accepted; cloud jobs pending
Restart server	P1 + P4	events and queue survive
Restore network + sync configured	P4	queued item becomes synced with real URL
GitHub disabled	P4 + P2	pending state, no fake link
New clone, no secrets	All	basic local demo works

13. Breaking-change procedure

1. Author opens `CONTRACT CHANGE` issue/message with before/after example and affected owners.
2. Affected owners approve; P3 edits this file and `shared/schemas.json` in the same PR.
3. P2/P4 update mocks and bridge; P1 updates service/storage where needed.
4. Run contract suite; only then merge.
5. During final 3 hours, **no breaking changes** unless the whole demo is blocked.

14. Demo fallback ladder

1. Live local coordinator + actual GitHub sync.
2. Live local coordinator + queued GitHub sync (no internet/token).
3. Live local coordinator + scripted simulator events.
4. If the server cannot run: static **clearly labeled** fixture walkthrough, not represented as working realtime software.

Single final success criterion: four separately owned modules integrate into a reproducible local-first collaboration demo with truthful offline/sync behavior and an auditable conflict workflow.